Abhishek Yadav

UsnNo: CS23071

Lab 4 Extra Datasets

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
path = "/content/drive/MyDrive/ML datasets/Developer_Performance_dataset_1000.csv"
df = pd.read_csv(path)
df.head()
```

| | Hours_Coding | Lines_of_Code | Bugs_Found | Bugs_Fixed | AI_Usage_Hours | Sleep_Hours | Cognitive_Load | Coffee_Intake | Stress_Level | Task_Dur |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 7 | 416 | 9 | 7 | 6 | 5.9 | 92 | 7 | 99 | |
| **1** | 4 | 269 | 16 | 13 | 5 | 5.1 | 85 | 2 | 100 | |
| **2** | 11 | 439 | 3 | 0 | 2 | 6.2 | 38 | 2 | 55 | |
| **3** | 8 | 472 | 15 | 9 | 4 | 4.2 | 26 | 5 | 30 | |
| **4** | 5 | 265 | 19 | 16 | 5 | 8.1 | 82 | 6 | 82 | |

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(1000, 13)
```

```
df.columns
```

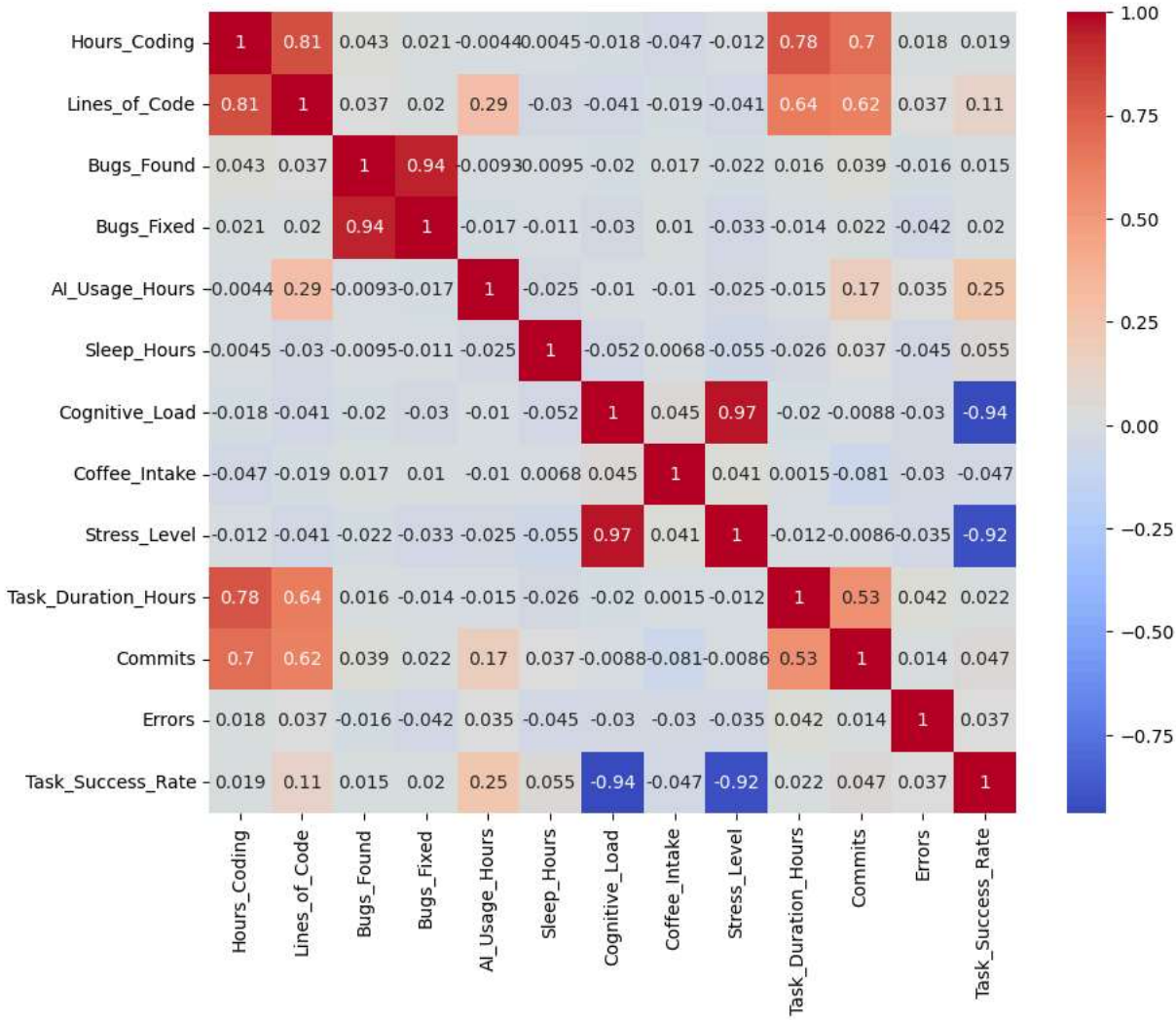
```
Index(['Hours_Coding', 'Lines_of_Code', 'Bugs_Found', 'Bugs_Fixed',
       'AI_Usage_Hours', 'Sleep_Hours', 'Cognitive_Load', 'Coffee_Intake',
       'Stress_Level', 'Task_Duration_Hours', 'Commits', 'Errors',
       'Task_Success_Rate'],
      dtype='object')
```

```
df.isnull().sum()
```

| | 0 |
|---|---|
| **Hours_Coding** | 0 |
| **Lines_of_Code** | 0 |
| **Bugs_Found** | 0 |
| **Bugs_Fixed** | 0 |
| **AI_Usage_Hours** | 0 |
| **Sleep_Hours** | 0 |
| **Cognitive_Load** | 0 |
| **Coffee_Intake** | 0 |
| **Stress_Level** | 0 |
| **Task_Duration_Hours** | 0 |
| **Commits** | 0 |
| **Errors** | 0 |
| **Task_Success_Rate** | 0 |

**dtype:** int64

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```




```
df["Productivity"] = df["Lines_of_Code"] / df["Hours_Coding"]
df.head()
```

| | Hours_Coding | Lines_of_Code | Bugs_Found | Bugs_Fixed | AI_Usage_Hours | Sleep_Hours | Cognitive_Load | Coffee_Intake | Stress_Level | Task_Dur |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 7 | 416 | 9 | 7 | 6 | 5.9 | 92 | 7 | 99 | |
| **1** | 4 | 269 | 16 | 13 | 5 | 5.1 | 85 | 2 | 100 | |
| **2** | 11 | 439 | 3 | 0 | 2 | 6.2 | 38 | 2 | 55 | |
| **3** | 8 | 472 | 15 | 9 | 4 | 4.2 | 26 | 5 | 30 | |
| **4** | 5 | 265 | 19 | 16 | 5 | 8.1 | 82 | 6 | 82 | |

Next steps: Generate code with df New interactive sheet

```
X = df.drop(columns=['Task_Success_Rate'])
y = df['Task_Success_Rate']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
df["Performance"] = df["Task_Success_Rate"].apply(lambda x: 1 if x >= 60 else 0)
```

```
X = df.drop(columns=['Task_Success_Rate', 'Performance'])
y = df['Performance']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

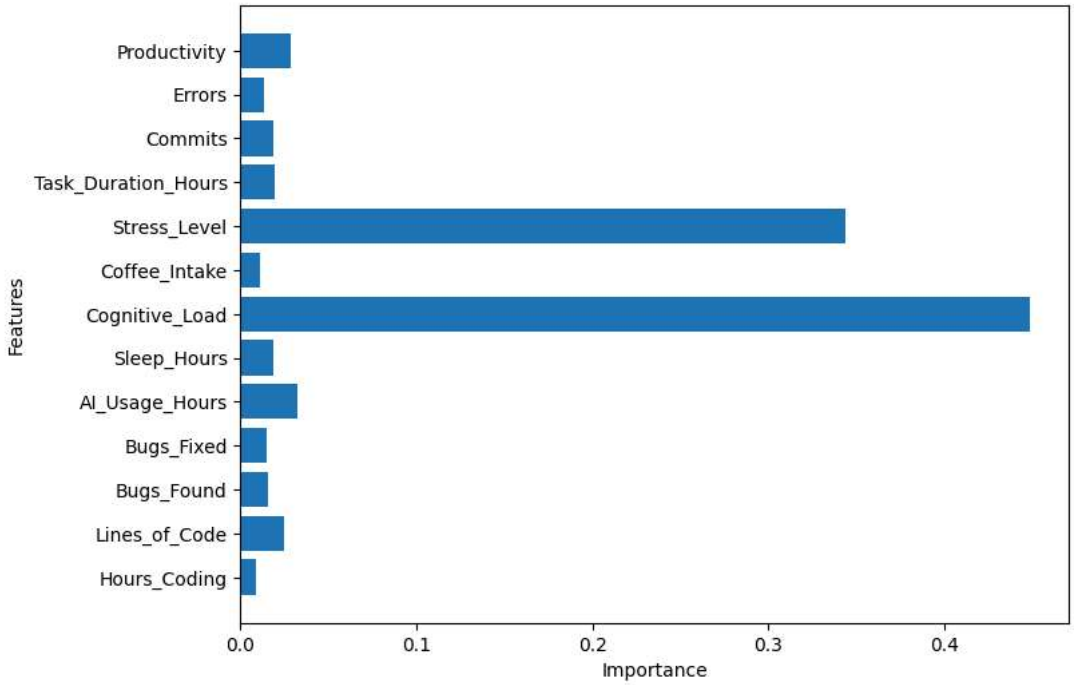
```
clf_model = RandomForestClassifier()
clf_model.fit(X_train, y_train)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
y_pred = clf_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.935

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.94      0.94       117
           1       0.92      0.93      0.92        83

    accuracy                           0.94       200
   macro avg       0.93      0.93      0.93       200
weighted avg       0.94      0.94      0.94       200
```

```
importances = clf_model.feature_importances_
features = X.columns

plt.figure(figsize=(8,6))
plt.barh(features, importances)
plt.xlabel("Importance")
plt.ylabel("Features")
plt.show()
```

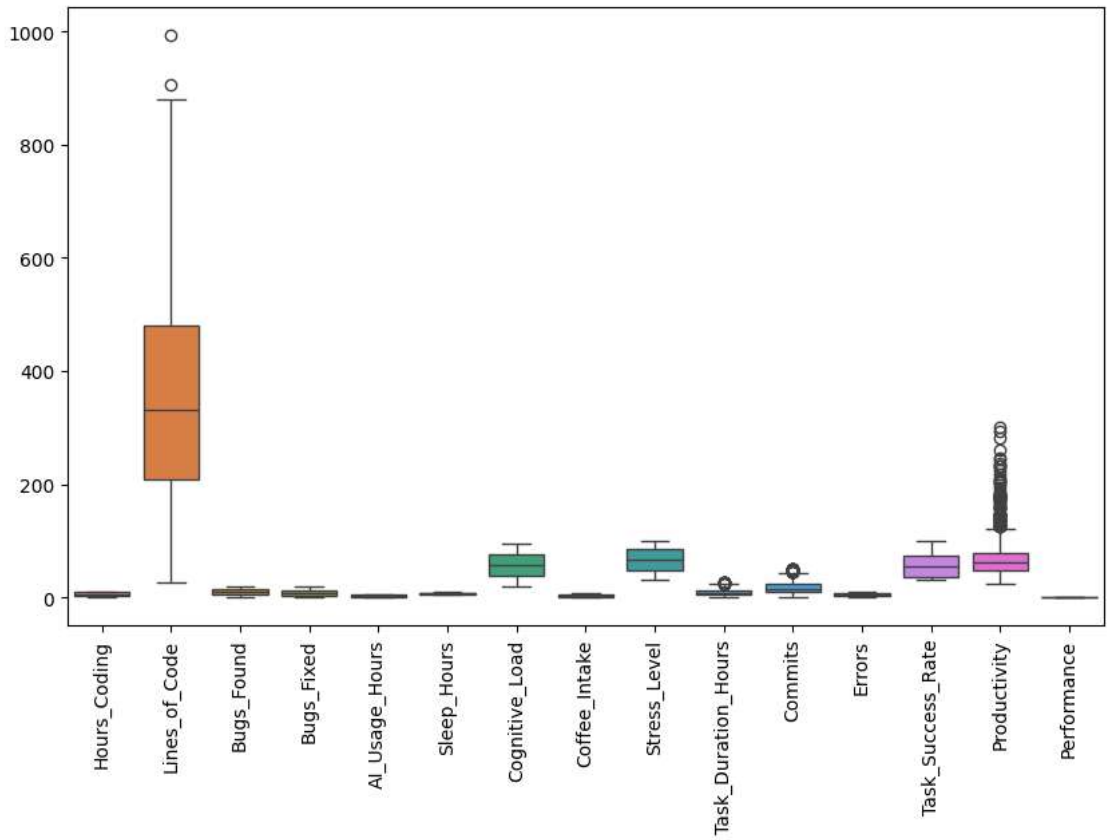



```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df = df.drop_duplicates()
```

```
plt.figure(figsize=(10,6))
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.show()
```



```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = df.drop(columns=['Task_Success_Rate'])
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
X_scaled.head()
```

| | Hours_Coding | Lines_of_Code | Bugs_Found | Bugs_Fixed | AI_Usage_Hours | Sleep_Hours | Cognitive_Load | Coffee_Intake | Stress_Level | Task_Dur |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.367442 | 0.317801 | -0.151213 | -0.027994 | 1.504257 | -0.393242 | 1.611785 | 1.551432 | 1.491225 | |
| **1** | -0.582839 | -0.463860 | 1.057110 | 1.069803 | 1.009273 | -0.949258 | 1.290044 | -0.584350 | 1.536982 | |
| **2** | 1.634484 | 0.440101 | -1.186918 | -1.308757 | -0.475680 | -0.184736 | -0.870218 | -0.584350 | -0.522089 | |
| **3** | 0.684202 | 0.615576 | 0.884492 | 0.337939 | 0.514288 | -1.574775 | -1.421774 | 0.697119 | -1.666017 | |
| **4** | -0.266079 | -0.485129 | 1.574963 | 1.618702 | 1.009273 | 1.135801 | 1.152155 | 1.124276 | 0.713354 | |

Next steps: Generate code with X_scaled New interactive sheet

```
df["Performance"] = df["Task_Success_Rate"].apply(lambda x: 1 if x >= 60 else 0)
X = df.drop(columns=['Task_Success_Rate','Performance'])
y = df['Performance']
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
y_pred = log_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.925
```

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
y_pred = dt_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.905
```

```
from sklearn.neural_network import MLPClassifier
ann_model = MLPClassifier(hidden_layer_sizes=(50,50), max_iter=500)
ann_model.fit(X_train, y_train)
y_pred = ann_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.92
```

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(clf_model, X, y, cv=5)
print("Cross Validation Accuracy:", scores.mean())
```

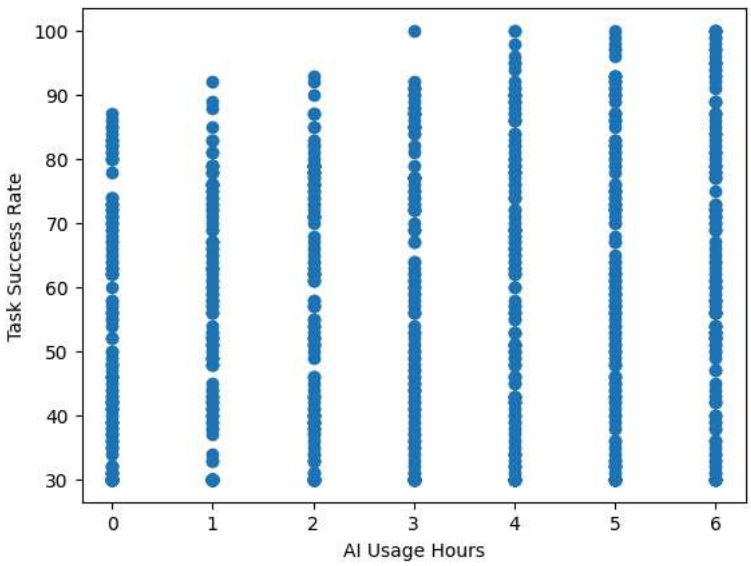
```
Cross Validation Accuracy: 0.93
```

```
models = {
    "Random Forest": clf_model,
    "Logistic Regression": log_model,
    "Decision Tree": dt_model,
    "ANN": ann_model
```

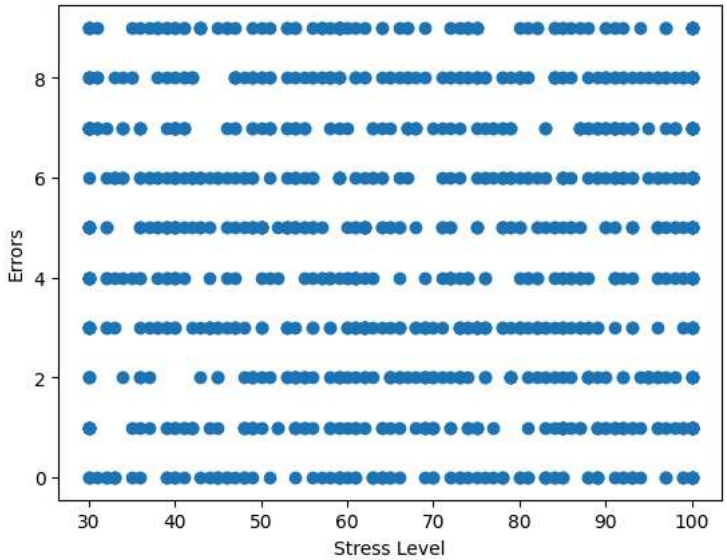
```
}

for name, model in models.items():
    y_pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, y_pred))
```

```
Random Forest Accuracy: 0.935
Logistic Regression Accuracy: 0.925
Decision Tree Accuracy: 0.905
ANN Accuracy: 0.92
```

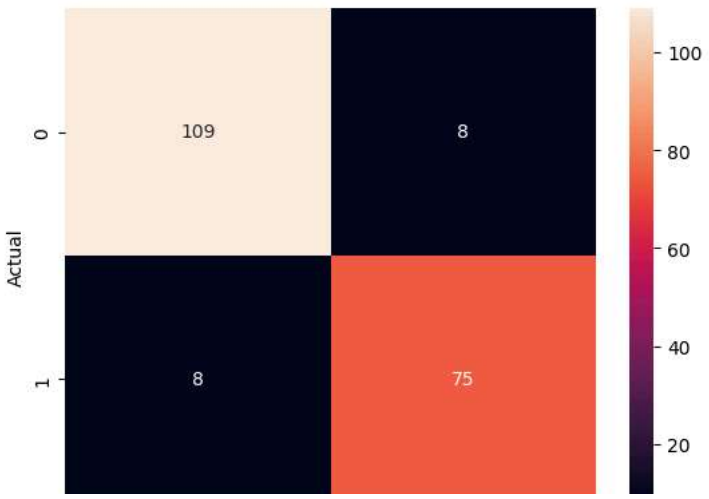
```
plt.scatter(df["AI_Usage_Hours"], df["Task_Success_Rate"])
plt.xlabel("AI Usage Hours")
plt.ylabel("Task Success Rate")
plt.show()
```




```
plt.scatter(df["Stress_Level"], df["Errors"])
plt.xlabel("Stress Level")
plt.ylabel("Errors")
plt.show()
```




```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.92

Classification Report:

              precision    recall  f1-score   support

           0       0.93      0.93      0.93       117
           1       0.90      0.90      0.90        83

    accuracy                           0.92       200
   macro avg       0.92      0.92      0.92       200
weighted avg       0.92      0.92      0.92       200
```

```
import seaborn as sns
import matplotlib.pyplot as plt
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```